Tugas 1: Analisis Struktur Aljabar & Fungsi Pemetaan

Pemetaan `str.maketrans()` dimodelkan sebagai fungsi $f : A \to B$ dengan $A = B = \{A, B, C, \ldots, Z\}$.

- Mengapa f harus bijektif

Dekripsi adalah fungsi $f^{-1} : B \to A$ dengan $f^{-1}(f(a)) = a$ untuk semua $a \in A$ dan $f(f^{-1}(b)) = b$ untuk semua $b \in B$.

Klaim: f^{-1} ada jika dan hanya jika f bijektif.

(⇒) Jika f^{-1} ada, maka f bijektif.

- *Injektif.* Misal $f(a_1) = f(a_2)$. Maka $a_1 = f^{-1}(f(a_1)) = f^{-1}(f(a_2)) = a_2$. Jadi $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$.
- *Surjektif.* Untuk sembarang $b \in B$, $b = f(f^{-1}(b))$, sehingga $f^{-1}(b) \in A$ adalah prapeta dari b.

(⇐) Jika f bijektif, maka f^{-1} ada. Karena f surjektif, setiap $b \in B$ punya paling sedikit satu prapeta. Karena f injektif, prapeta itu tunggal. Definisikan $f^{-1}(b) = a$ dengan $f(a) = b$. Maka f^{-1} terdefinisi pada seluruh B dan bernilai tunggal, sehingga merupakan fungsi.

Karena $|A| = |B| = 26$ (himpunan hingga), injektif $\iff$ surjektif $\iff$ bijektif. Jadi f harus berupa permutasi dari A.

- Dampak jika f non-injektif

`random.shuffle()` hanya menukar posisi elemen yang sudah ada, sehingga hasilnya selalu permutasi dan tidak pernah memunculkan elemen berulang. Pertanyaan ini bersifat pengandaian, misalnya kunci diketik manual dan ada huruf kembar. Jika $f(a_1) = f(a_2) = c$ dengan $a_1 \neq a_2$:

- $f^{-1}(c)$ tidak bernilai tunggal, sehingga f^{-1} bukan fungsi. Ciphertext c bisa berasal dari a_1 atau a_2, dekripsi menjadi ambigu.
- $|f(A)| < 26$, jadi ada huruf di B yang tidak pernah muncul sebagai ciphertext (f tidak surjektif).
- Pesan berbeda menghasilkan ciphertext yang sama (informasi hilang permanen).
- Jika ada m posisi ambigu pada ciphertext (masing-masing 2 kandidat), jumlah plaintext yang mungkin adalah 2^m.
- Pada kode, `str.maketrans` membangun dictionary sehingga kunci yang kembar ditimpa entri terakhir. Dekripsi selalu memilih satu kandidat saja.

Simulasi di bawah ini memakai kunci cacat: `A` dan `B` sama-sama dipetakan ke `A`.

```
import string

alphabet = string.ascii_uppercase
bad_key = "AA" + alphabet[2:]          # A -> A dan B -> A (non-injektif)

enc_table = str.maketrans(alphabet, bad_key)
dec_table = str.maketrans(bad_key, alphabet)

pesan = "AB BA AA BB"
cipher = pesan.translate(enc_table)
hasil = cipher.translate(dec_table)

print("Plaintext      :", pesan)
print("Ciphertext     :", cipher)
print("Hasil dekripsi :", hasil)
print("Sama dengan plaintext?", hasil == pesan)
print("Jumlah huruf berbeda di kunci:", len(set(bad_key)), "(seharusnya 26)")
```

```
Plaintext      : AB BA AA BB
Ciphertext     : AA AA AA AA
Hasil dekripsi : BB BB BB BB
Sama dengan plaintext? False
Jumlah huruf berbeda di kunci: 25 (seharusnya 26)
```

Tugas 2: Analisis Kombinatorika & Ruang Kunci

- Ruang kunci

`generate_key()` mengacak 26 huruf yang berbeda, yaitu permutasi dari 26 objek:

$$|K| = P(26, 26) = 26! = 26 \times 25 \times 24 \times \cdots \times 2 \times 1$$

- Waktu brute force

Dengan kecepatan 10^{12} kunci per detik:

$$t = \frac{26!}{10^{12}} \text{ detik}, \qquad 1 \text{ tahun} = 365 \times 24 \times 3600 = 31.536.000 \text{ detik}$$

Perhitungannya dijalankan di bawah ini.

```
import math

ruang_kunci = math.factorial(26)
kecepatan = 10**12                      # kunci per detik
detik_per_tahun = 365 * 24 * 3600       # 31.536.000

total_detik = ruang_kunci / kecepatan
tahun_penuh = total_detik / detik_per_tahun
tahun_rata2 = tahun_penuh / 2           # kunci benar rata-rata ketemu di tengah

print(f"Ruang kunci 26!      = {ruang_kunci:,}")
print(f"                     = {ruang_kunci:.6e}")
print(f"Total waktu (detik)  = {total_detik:.6e}")
print(f"Seluruh ruang kunci  = {tahun_penuh:.4e} tahun (sekitar {tahun_penuh/1e6:.1f} juta tahun)")
print(f"Rata-rata (setengah) = {tahun_rata2:.4e} tahun (sekitar {tahun_rata2/1e6:.1f} juta tahun)")
```

```
Ruang kunci 26!      = 403,291,461,126,605,635,584,000,000
                     = 4.032915e+26
Total waktu (detik)  = 4.032915e+14
Seluruh ruang kunci  = 1.2788e+07 tahun (sekitar 12.8 juta tahun)
Rata-rata (setengah) = 6.3941e+06 tahun (sekitar 6.4 juta tahun)
```

Tugas 3: Kriptanalisis Statistik & Rekonstruksi Teks

- Mengapa sistem tetap tidak aman

Kunci tidak perlu ditebak satu per satu. Enkripsi monoalfabetik memetakan setiap huruf plaintext ke satu huruf ciphertext yang sama di seluruh pesan. Akibatnya distribusi frekuensi huruf bahasa asli terbawa utuh ke ciphertext, hanya labelnya yang berubah.

- Huruf paling sering di ciphertext hampir pasti pasangan huruf paling sering di bahasa Indonesia (A, N, I, E, ...).
- Penyerang mencocokkan peringkat frekuensi, lalu memperbaikinya dengan pola bigram/trigram (misalnya `NG`, `AN`, `KAN`), panjang dan pola kata (spasi dipertahankan oleh kode), serta konteks.
- Pencarian $26!$ kunci turun menjadi beberapa tebakan terarah.

Kelemahannya ada pada struktur (satu huruf selalu menjadi satu huruf), bukan pada ukuran ruang kunci.

- Rekonstruksi ciphertext `VQZRVQZOSQ`

Langkah pertama adalah menghitung frekuensi setiap karakter.

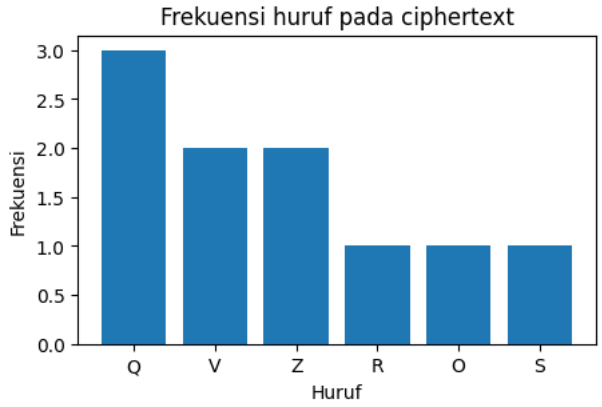
```
from collections import Counter
import matplotlib.pyplot as plt

ciphertext = "VQZRVQZOSQ"
frek = Counter(ciphertext)
n = len(ciphertext)

print(f"{'Huruf':<7}{'Frekuensi':<11}{'Persentase'}")
for huruf, jumlah in frek.most_common():
    print(f"{huruf:<7}{jumlah:<11}{jumlah / n * 100:.0f}%")

huruf, jumlah = zip(*frek.most_common())
plt.figure(figsize=(5, 3))
plt.bar(huruf, jumlah)
plt.title("Frekuensi huruf pada ciphertext")
plt.xlabel("Huruf")
plt.ylabel("Frekuensi")
plt.show()
```

```
Huruf  Frekuensi  Persentase
Q      3          30%
V      2          20%
Z      2          20%
R      1          10%
O      1          10%
S      1          10%
```



Langkah rekonstruksi manual

1. `Q` paling sering muncul (3 kali, pada posisi 2, 6, dan 10), sesuai asumsi soal, maka $Q = A$.
2. Potongan `VQZ` muncul dua kali, jadi ada suku kata atau kata yang berulang. Dengan $Q = A$ polanya menjadi `?A??` `?A?` `???A`.
3. Teks 10 huruf berpola ABCD ABC EFA yang masuk akal dalam bahasa Indonesia adalah **MATEMATIKA** (M-A-T-E-M-A-T-I-K-A).
4. Pemetaan yang konsisten: $V = M,\ Q = A,\ Z = T,\ R = E,\ O = I,\ S = K$.

Catatan: sampel hanya 10 huruf, jadi frekuensi saja belum cukup untuk memastikan. Kuncinya ada pada pola pengulangan `VQZ` dan tebakan kata. Pada teks yang panjang, frekuensi saja sudah sangat kuat.

```
pemetaan = {"V": "M", "Q": "A", "Z": "T", "R": "E", "O": "I", "S": "K"}
plaintext = "".join(pemetaan[c] for c in ciphertext)

print("Ciphertext:", ciphertext)
print("Plaintext :", plaintext)
```

Tugas 4: Modifikasi & Pengembangan Kode

Ketentuan:

- Huruf alfabet dienkripsi dengan substitusi acak (huruf besar dan kecil dipertahankan).
- Angka dan simbol matematika `0123456789+-=*` dipetakan ke posisi terbaliknya (reverse order), yaitu `*=-+9876543210`. Jadi `0↔*`, `1↔=`, `2↔-`, `3↔+`, `4↔9`, `5↔8`, `6↔7`.
- Karakter lain (spasi, koma, tanda seru, dan sebagainya) tidak diubah.

Mengapa `decrypt()` adalah invers sah dari `encrypt()`

Domain tabel enkripsi adalah gabungan tiga himpunan yang saling lepas: huruf besar, huruf kecil, dan `NUM_SYM`. Pada tiap bagian pemetaannya berupa permutasi dari himpunan itu sendiri (kunci acak untuk huruf, pembalikan untuk angka dan simbol). Gabungan bijeksi pada himpunan yang saling lepas adalah bijeksi, sehingga inversnya diperoleh dengan menukar argumen `maketrans`.

```
import string
import random

ALPHABET = string.ascii_uppercase
NUM_SYM = "0123456789+-=*"        # angka dan simbol matematika
NUM_SYM_REV = NUM_SYM[::-1]       # urutan terbalik (reverse order)


def generate_key():
    shuffled = list(ALPHABET)
    random.shuffle(shuffled)
    return ALPHABET, "".join(shuffled)


def encrypt(plaintext, key_map):
    alphabet, cipher_key = key_map
    # huruf: substitusi acak | angka dan simbol: dipetakan ke posisi terbalik
    table = str.maketrans(
        alphabet + alphabet.lower() + NUM_SYM,
        cipher_key + cipher_key.lower() + NUM_SYM_REV,
```

```
    )
    return plaintext.translate(table)


def decrypt(ciphertext, key_map):
    alphabet, cipher_key = key_map
    # invers: argumen maketrans ditukar
    table = str.maketrans(
        cipher_key + cipher_key.lower() + NUM_SYM_REV,
        alphabet + alphabet.lower() + NUM_SYM,
    )
    return ciphertext.translate(table)
```

```
# Pengujian
alphabet, key = generate_key()
pesan = "Hitung 12+34=46, lalu 5*6-7 = 23 di Kriptografi!"
cipher = encrypt(pesan, (alphabet, key))
plain = decrypt(cipher, (alphabet, key))

print("Plaintext :", pesan)
print("Kunci Acak:", key)
print("Ciphertext:", cipher)
print("Decrypted :", plain)
print("Valid     :", plain == pesan)
```

```
Plaintext : Hitung 12+34=46, lalu 5*6-7 = 23 di Kriptografi!
Kunci Acak: RTSYMWGDXJOEVNQPBIAKHFZLUC
Ciphertext: Dxkhng =-3+9197, ereh 80726 1 -+ yx Oixpkqgirwx!
Decrypted : Hitung 12+34=46, lalu 5*6-7 = 23 di Kriptografi!
Valid     : True
```

```
# Uji invers pada 2000 kunci dan teks acak
karakter = string.ascii_letters + NUM_SYM + " ,.!/()"
for _ in range(2000):
    k = generate_key()
    s = "".join(random.choice(karakter) for _ in range(random.randint(0, 60)))
    assert decrypt(encrypt(s, k), k) == s
print("decrypt(encrypt(x)) == x untuk 2000 uji acak: LULUS")

# Tabel pemetaan angka dan simbol
print("\nPemetaan angka dan simbol:")
print("  ".join(f"{a}->{b}" for a, b in zip(NUM_SYM, NUM_SYM_REV)))
```

```
decrypt(encrypt(x)) == x untuk 2000 uji acak: LULUS

Pemetaan angka dan simbol:
0->*  1->=  2->-  3->+  4->9  5->8  6->7  7->6  8->5  9->4  +->3  -->2  =->1  *->0
```